

Midterm Mastery:

Distributed Systems in Practice

Sihui Lyu
CS6650 – Distributed Systems

Presentation Overview

Part 1

- Scaling Limits & Bottlenecks (HW6)

→ Vertical vs Horizontal Scaling

Part 2

- Failure & Resilience (Crash & Recover)

→ Fail Fast / Bulkhead / Circuit Breaker

Part 3

- What Changed My Thinking

→ Distributed Systems Mental Model

Part 1

Vertical Scaling Limit (Problem)

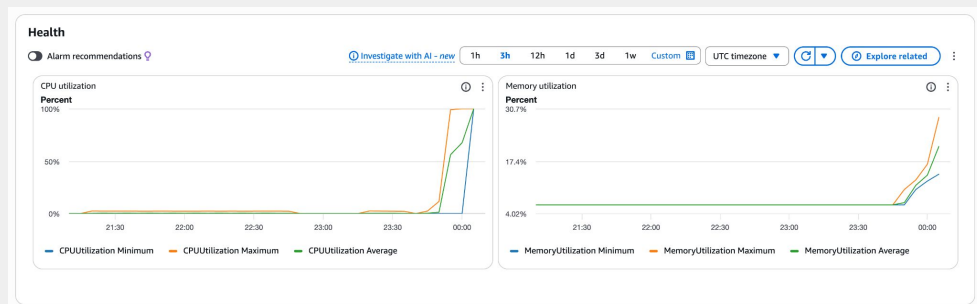
Load Testing: Product Search API on ECS (50 → 5000 users)

Key Observations:

- Throughput increased initially but began to plateau at high load.
- Average latency rose gradually and then sharply at peak load (~5000 users).
- CPU utilization reached ~100%, indicating a compute-bound bottleneck.
- Even without failures, performance degradation was visible via increased latency and throughput flattening.

Conclusion:

CPU saturation marked the vertical capacity limit.

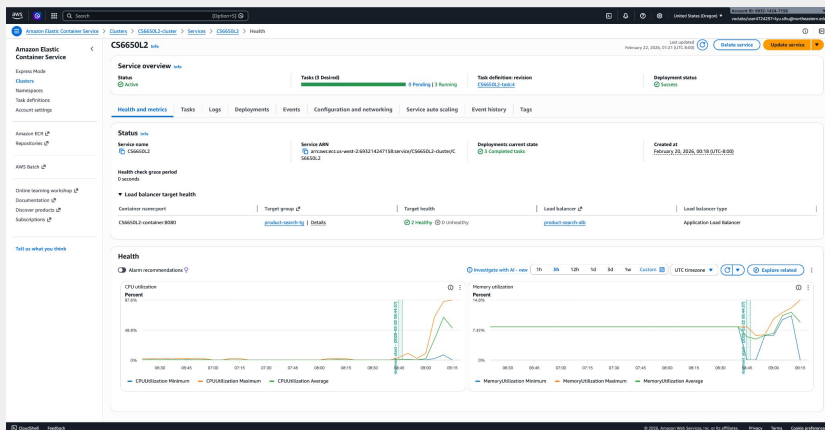


Horizontal Scaling (Solution)

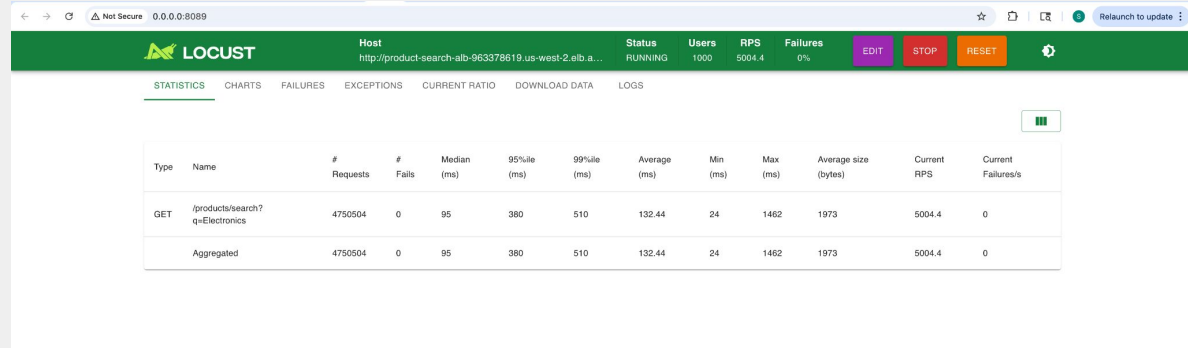
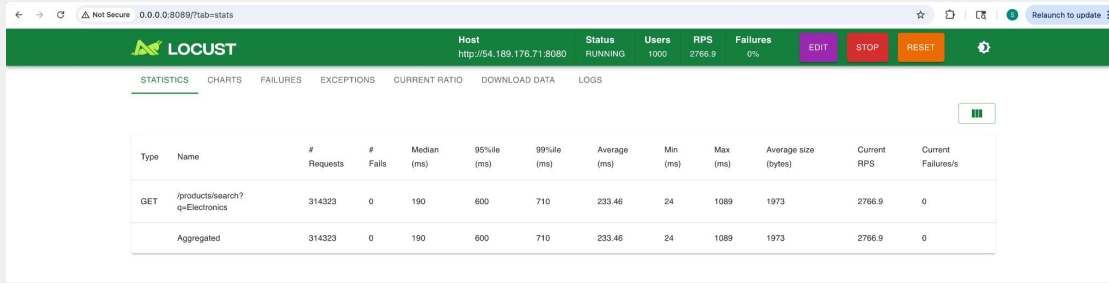
ALB + ECS Service Auto Scaling

Key Observations:

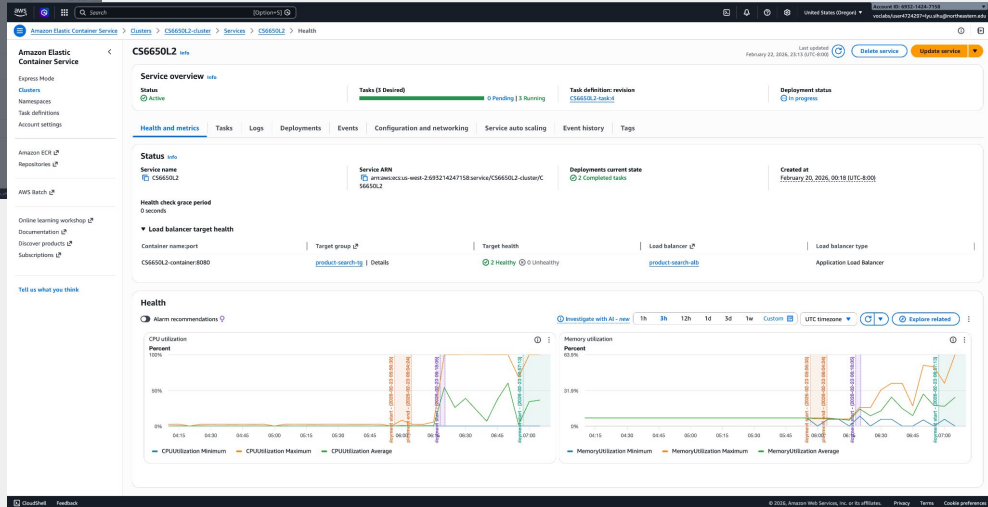
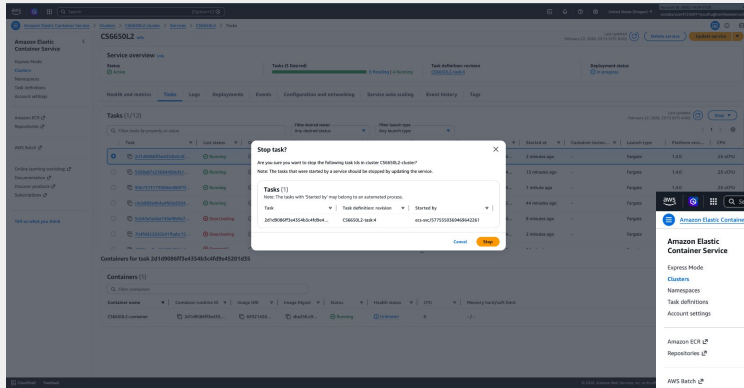
- Under higher load (~1000 users), additional ECS tasks were automatically provisioned as average CPU utilization exceeded the scaling threshold.



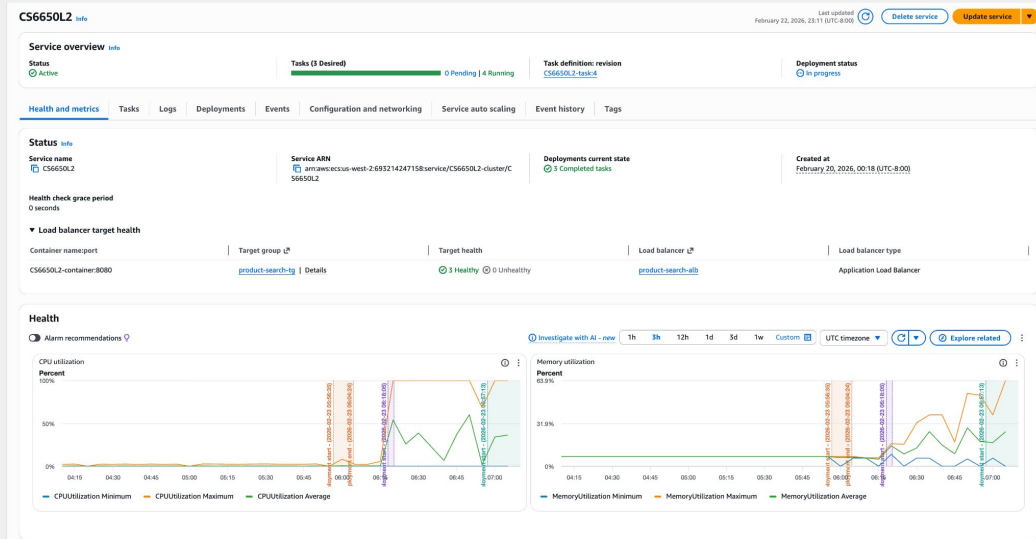
- Average response time improved compared to Part II because requests were processed in parallel across multiple tasks, reducing queueing delay.



- Stopping a running task did not impact service availability because remaining tasks continued serving traffic through the load balancer



- Lowering the threshold to 40% allowed scaling to occur earlier under the same workload, demonstrating increased scaling sensitivity.



- One interesting finding was that identical workload levels sometimes produced different scaling outcomes. This appears to be caused by stochastic load balancer request distribution and scaling decisions based on averaged CPU metrics rather than user count.

Part 2

Intentionally Breaking the System

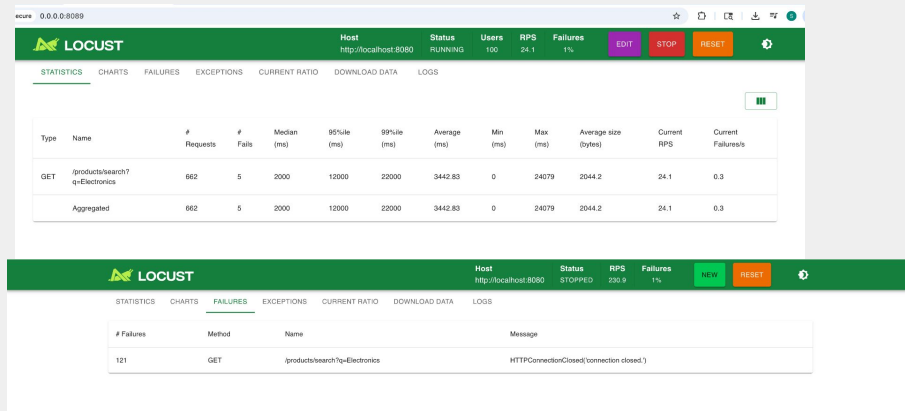
Crash Scenario — Cascading Failure

Injected a blocking downstream dependency:

- 2-second latency
- No timeout
- No bulkhead
- No circuit breaker

Results under 100 users:

- Median latency: 2000 ms
- P99 latency: 22,000 ms
- HTTPConnectionClosed errors
- Throughput $\approx$ 24 RPS



This demonstrates latency amplification under blocking calls.

Recovery Strategy 1: Fail Fast

Timeout = 300 ms

Results:

- Median latency: 300 ms
- P95 latency: 300 ms
- Throughput $\approx$ 476 RPS
- 0% failures

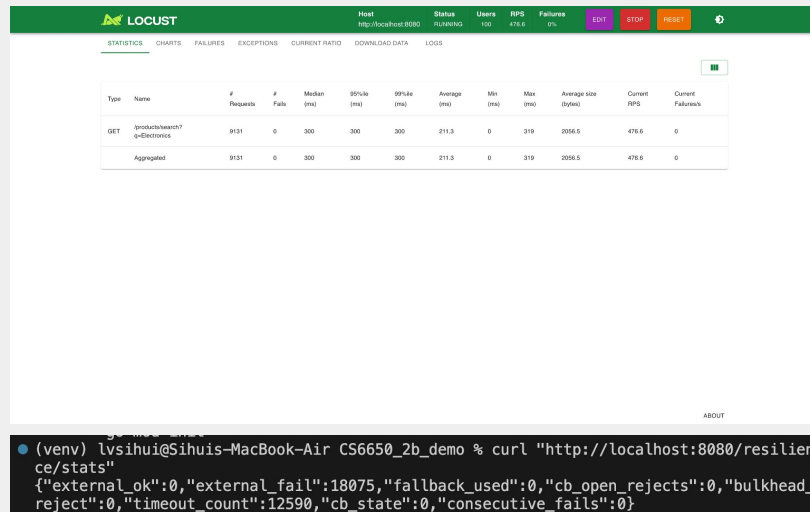
Compared to crash scenario:

- P99 reduced from 22 seconds $\rightarrow$ 300 ms

Why it works:

- Requests terminate early
- Resources freed quickly
- Tail latency eliminated

Fail-fast protects latency.



Recovery Strategy 2: Bulkhead Isolation

MaxConcurrent = 50

Results:

- Median latency $\approx$ 1 ms
- RPS $\approx$ 24,000
- No failures

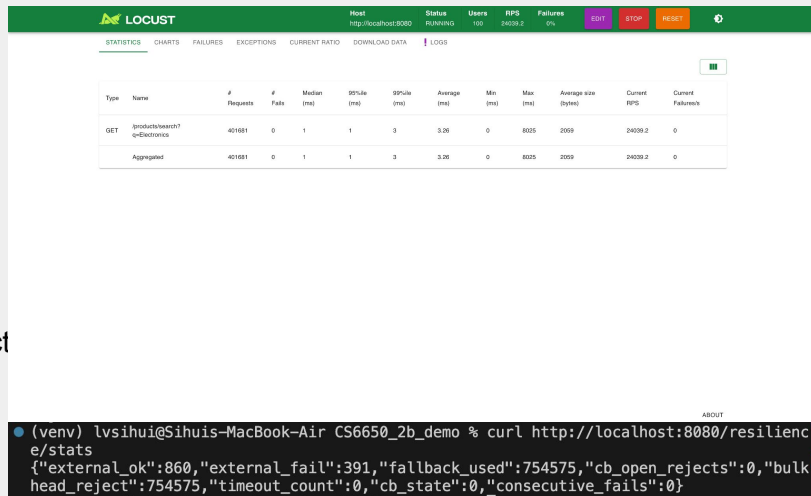
Compared to crash scenario:

- Instead of queuing, excess requests reject

Why it works:

- Limits concurrent downstream calls
- Prevents resource exhaustion

Bulkhead protects system resources.



Recovery Strategy 3: Circuit Breaker

FailThreshold = 20

Results:

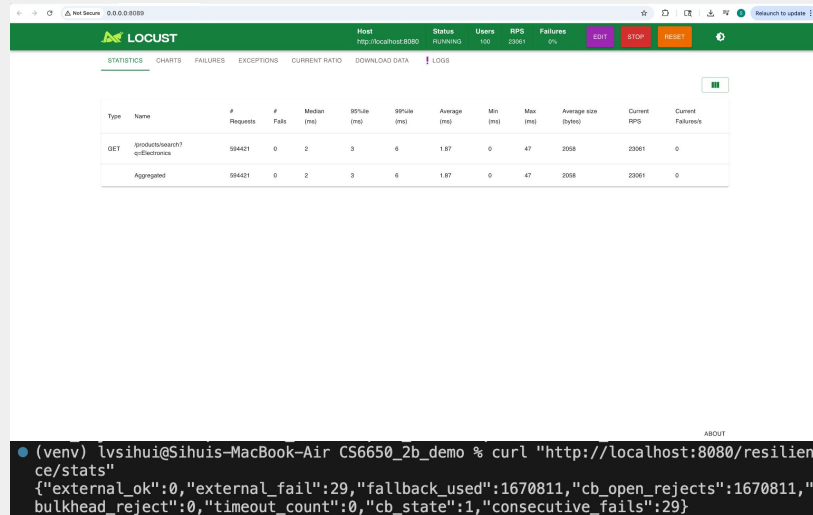
- Circuit transitioned CLOSED → OPEN
- Downstream calls stopped
- Median latency $\approx$ 1–2 ms
- RPS $\approx$ 23,000

Internal metrics:

- cb_state: OPEN
- cb_open_rejects: 1.6M+

Why it works:

- Detects dependency failure state
- Proactively cuts traffic



Circuit breaker protects against unhealthy dependencies.

Comparative Summary

Strategy	What It Protects	Effect
None	Nothing	Cascading failure
Fail Fast	Latency	Bounds request duration
Bulkhead	Resources	Limits concurrency
Circuit Breaker	Failure state	Stops unhealthy traffic

Key Insight:

Resilience is about bounding failure, not eliminating it.

Part 3

What I Found Most Valuable

- Hands-on AWS deployment & load testing
- Seeing CPU saturation in real experiments
- Understanding CAP trade-offs
- Connecting concurrency primitives to system-level behavior
- Learning resilience patterns in practice

Reflection on Learning Progress in Distributed Systems

Since the beginning of this course, I have gained substantial knowledge about distributed systems, a field that was largely unfamiliar to me due to my non-distributed systems background. This learning journey has been particularly meaningful because it fundamentally changed how I perceive modern software systems. Previously, I tended to think about systems as single-machine applications with relatively straightforward execution models. Through this course, I began to understand the inherent complexity of distributed environments and the design trade-offs required to build scalable and reliable systems.

One of the most valuable aspects of the course has been the evidence-based experience with cloud-based infrastructure and distributed deployments, in particular, working with Go provided exposure to a language commonly used in distributed system development due to its lightweight concurrency model. More importantly, the AWS-based assignments allowed me to experience the full lifecycle of deploying and operating a distributed service. For example, in a recent assignment, we deployed a small web service using Amazon ECS and conducted load testing using Locust. By gradually increasing the number of users to approximately 5000, we observed system behavior under stress, including CPU utilization reaching saturation levels.

This experiment provided a concrete understanding of performance bottlenecks and capacity limits, concepts that are often abstract in lectures but become tangible through experimentation. Building on this experience, we further explored horizontal scaling strategies using load balancers and auto-scaling mechanisms. Observing how additional service instances could be provisioned dynamically in response to increased demand helped me develop an intuition for elasticity in cloud-native systems. Rather than viewing scaling as a theoretical concept, I was able to see how infrastructure components cooperate to maintain service availability and performance.

Users → Load Balancer → EC2 Instance
Load Balancer → CPU Utilization → Auto Scaling → EC2 Instance 1,2,3

In addition to practical experimentation, theoretical topics discussed in class and readings also shaped my understanding of distributed system design. The MapReduce paper was particularly memorable because it demonstrated how large-scale data processing tasks can be decomposed and executed across clusters of machines in a fault-tolerant manner. This reading illustrated how distributed computation frameworks abstract complexity while enabling massive scalability, reinforcing the importance of system design patterns that support parallel execution.

Map → Shuffle → Reduce → Output

Another key concept that significantly influenced my thinking was the CAP theorem. Learning that distributed systems must navigate trade-offs between consistency, availability, and partition tolerance challenged my previous assumption that systems could simultaneously optimize all desirable properties. Understanding CAP provided a mental framework for reasoning about real-world system behavior, especially during failures or network partitions, and highlighted why different systems make different architectural choices.



The course also introduced essential tooling commonly used in modern distributed development workflows. Learning Docker and Terraform demonstrated how infrastructure and application environments can be standardized and reproduced across machines. Containerization clarified how developers can ensure consistent runtime behavior regardless of local setup differences, while Terraform emphasized the declarative management of infrastructure resources. Together, these tools facilitated how distributed system engineering extends beyond application logic into deployment automation and infrastructure orchestration.

Finally, concurrency concepts explored through threading and load testing exercises further deepened my understanding of system behavior under parallel workloads. Studying synchronization primitives such as Mutex and RWMutex provided insight into how concurrent access to shared resources can be coordinated efficiently. These experiments helped connect language-level concurrency constructs with system-level performance considerations, reinforcing the importance of careful design in multi-threaded environments.

Overall, the combination of theoretical foundations, research readings, peer discussions, and hands-on cloud experimentation has made this course highly engaging and valuable. It not only introduced me to distributed system concepts but also reshaped my mental model of how modern applications operate at scale. I now approach system design with greater awareness of trade-offs, failure modes, and scalability strategies, and I feel distributed systems to be an intellectually stimulating and practically relevant area of study.

Final Takeaway:

Distributed systems are defined by trade-offs, failure modes, and scaling strategies — not just application logic.

Thanks!